Neurofeedback fNIRS (NIRx/Aurora + PsychoPy) — Documentação técnica

Neurofeedback fNIRS (NIRx/Aurora + PsychoPy) — Documentação técnica

Ferramenta de pesquisa para neurofeedback em tempo real com fNIRS (NIRSport2 + Aurora), apresentação em PsychoPy.

Convenções: “a confirmar” = o código não permite afirmar ou o dado depende de verificação em campo. Tudo o mais descreve o que o código faz (revisão dos scripts da pasta do projeto).

O sistema foi inspirado, quanto à arquitetura modular (threads/janelas separadas para stream, processamento e visualização), em um sistema de neurofeedback EEG de referência (repositório `Neurofeedback-with-MNE-and-PsychoPy`). Nenhum código dele está copiado neste projeto; a camada de sinal (ASR/ICA, PSD) não se aplica a fNIRS e foi substituída.

1. Visão geral e arquitetura

1.1 As 5 camadas

#	Camada	Implementação
1	Paradigma / markers	Fases controladas por `nfb_process.py` (baseline → regulação → stop); markers publicados no stream LSL `Trigger`
2	Aquisição	`acquisition.py` (`pylsl`, mapeamento de colunas pela metadata)
3	Processamento	`processing.py` (`FeaturePipeline`)
4	Visualização	`trail_display.py` (`TrailDisplay`, PsychoPy)
5	Log	`nf_logger.py` (`Logger`, CSV + JSON de metadados)

O arquivo `Fingertapping_LSL.psyexp` (teste de sincronização, PsychoPy Builder) é referência de como o trigger é emitido; não faz parte do código desta pasta e não é a tarefa final.

1.2 Os dois processos

```
PROCESSO A                                  PROCESSO B
control_panel.py (tkinter)                  nfb_process.py (PsychoPy)
painel flutuante, 2o monitor                janela de neurofeedback
       |                                          |
       |   run/nfb_command.json  (comandos)       |
       |  ---------------------------------------->   |  le a 30 Hz
       |                                          |
       |   run/nfb_state.json    (estado)         |
       |  <----------------------------------   |  escreve a cada tick
       |  le a cada 300 ms                        |
       +----------- session_ipc.py ----------------+
        (escrita atomica tmp + os.replace, retry)
```

O painel (A) só fala LSL diretamente na aba Configurações (“Detectar canais”), com um inlet independente do que o processo B abre na sessão. Toda a sessão é conduzida por comandos (`CONFIGURE`, `SYNC`, `PLAY`, `START_NFB`, `STOP`) enviados

por arquivo.

1.3 Fluxo de dados ponta a ponta

```
NIRSport2 -> Aurora --(LSL Data Out, nome "Aurora")--> acquisition.py
                                                       |  StreamMap (colunas pela metadata)
                                                       |  Acquisition.pull/drain (descarta NaN)
                                                       v
                                          SignalWorker (thread, run_neurofeedback.py)
                                                       |  add_sample(device_time, HbO dos canais do ROI)
                                                       v
                                          processing.py  FeaturePipeline
                                        media ROI -> janela -> z-score(baseline) -> clip -> EMA
                                                       |  altitude (unidades ALTITUDE_RANGE)
                                                       v
                                          trail_display.py  TrailDisplay
                                          (progresso 0-1 na trilha, suavizacao adicional)
                                                       |
                                          nf_logger.py <--+  uma linha por tick (~30 Hz)

nfb_process.py --(LSL, StreamOutlet "Trigger", tipo Markers, int32)--> Aurora (Trigger In)
  codigos: 1 = inicio baseline | 2 = inicio regulacao | 3 = stop
```

O `Trigger` é publicado pelo `nfb_process.py` desde a criação de `NfbSession` (não só em PLAY), para que o passo SYNC encontre um stream `Markers` sem depender de outro processo.

2. Scripts

Formato: propósito; entradas/saídas; classes e funções principais; quem chama; dependências.

2.1 Núcleo

`nf_config.py`

- **Propósito:** parâmetros centrais. Sem lógica.
- **Entradas/saídas:** constantes lidas por quase todos os módulos. Define `BASE_DIR`, `LOG_DIR` (`logs/`), `RUN_DIR` (`run/`).
- **Quem chama:** todos. **Dependências:** `os`.
- Tabela completa na seção 3.

`acquisition.py` (Camada 2)

- **Propósito:** conectar ao stream LSL Data Out e mapear colunas pela metadata (nunca por índice fixo).
- **Entradas:** nome do stream (`cfg.STREAM_NAME` ou argumento), lista de pares `(fonte, detector)` do ROI. **Saídas:** tuplas `(lsl_ts, device_ts, vetor_HbO_ROI)`.
- `_translate_type(raw_type, wavelength)` — traduz `type` da metadata real (`nirs_hbo` → `HbO` etc.; `nirs_raw` + `wavelength` → `raw760`/`raw850`).
- `_parse_source_detector(custom_name, label)` — extrai `S{n}`/`D{n}` de `custom_name`, com fallback no `label`.
- `StreamMap` — índice de colunas a partir de `StreamInfo`; métodos `columns`, `pfc_hbo_columns`, `short_hbo_columns`, `timestamp_column`. Levanta `RuntimeError` se canais do ROI estiverem ausentes ou a metadata for incompleta.
- `Acquisition` — resolve o stream por nome (`resolve_byprop("name", ...)`), cria o inlet (`max_buflen=60`, `recover=False`); `pull`, `drain`, `close`. Amostra com qualquer NaN/Inf nos canais do ROI é descartada (`n_invalid`) e nunca sobrescreve a última válida.
- **Quem chama:** `nfb_process.py`, `run_neurofeedback.py`, `control_panel.py` (só `StreamMap`), testes. **Dependências:** `pylsl`, `numpy`, `nf_config`.

`processing.py` (Camada 3)

- **Propósito:** feature = média de HbO dos canais do ROI → janela deslizante → z-score pelo baseline → saturação → suavização exponencial.
- `FeaturePipeline` — estados internos `baseline`, `baseline_ready` (só se `manual_gate=True`), `running`, `failsafe`. Métodos: `add_sample(t, pfc_hbo)`, `begin_regulation()`, `update_display(now)`, `mark_sample_received(wall_time)`, `snapshot()`.
- **Quem chama:** `SignalWorker`, `NfbSession`. **Dependências:** `numpy`, `nf_config`.

`trail_display.py` (Camada 4)

- **Propósito:** explorador sobre trilha de montanha; progresso (0–1) derivado da altitude do pipeline.
- `position_on_trail(progress, points)` — interpola linearmente entre os 6 waypoints (`TRAIL_POINTS_NORMALIZED`, convertidos de coordenadas canto-superior-esquerdo para o sistema centrado do PsychoPy).
- `TrailDisplay` — janela (`visual.Window`, 1280×720 por padrão), fundo `trail_bg.png`, sprite `hiker_sprite.png` ancorado nos pés (`bottom-center`), texto de status e aviso; `draw(altitude, snap, dt)`, `should_quit()` (tecla Esc), `close()`. Aplica suavização adicional sobre o progresso (`SMOOTHING_SPEED = 1.2`).
- **Quem chama:** `NfbSession.ensure_display`, `run_neurofeedback.run`. **Dependências:** `psychopy`, `PIL`, `nf_config`. Os PNGs são lidos da mesma pasta do script.

`nf_logger.py` (Camada 5)

- **Propósito:** CSV por sessão + JSON de metadados.
- `Logger(prefix, meta, log_dir)` — cria `logs/<prefix>_<AAAA-MM-DD_HHMMSS>.csv` e `..._meta.json`; `write(...)` grava uma linha; `close()`.
- Colunas do CSV: `wall_iso, wall_unix, lsl_time, device_time, state, raw_feature_hbo, windowed_hbo, normalized, smoothed, altitude, valid, n_valid, n_invalid, marker`. `lsl_time/device_time` permitem reconciliar com o `*_lsl.tri` do Aurora (procedimento de reconciliação: a confirmar).
- **Quem chama:** `NfbSession`, `run_neurofeedback.run`, testes. **Dependências:** `csv`, `json`, `datetime`.

2.2 Aplicação de produção (dois processos)

`nfb_process.py` (Processo B)

- **Propósito:** janela de neurofeedback, orquestração da sessão, publicação do `Trigger`.
- `NfbSession` — máquina de estados por fase (seção 5). `handle_command`, `_do_sync`, `_do_play`, `_do_start_nfb`, `_do_stop`, `_emit_marker`, `tick`, `_publish_state`, `run_forever(poll_hz=30)`. A lógica é isolada do display (aceita `display_factory`), permitindo teste com display falso.
- Constante `PLAUSIBLE_HBO_ABS_MAX = 200.0` µmol/L (checagem de qualidade no SYNC).
- Ao executar como script: `ipc.reset()` e `NfbSession().run_forever()`.
- **Dependências:** `pylsl`, `session_ipc`, `acquisition`, `processing`, `nf_logger`, `run_neurofeedback.SignalWorker`, `trail_display` (import adiado).

`control_panel.py` (Processo A)

- **Propósito:** painel tkinter (abas Principal e Configurações).
- `ControlPanel` — config: nome do stream, "Detectar canais" (thread com `resolve_byprop + StreamMap`, lista pares com tipo `HbO`), seleção do ROI por checkboxes (pré-marcados: `PFC_CHANNELS`), ID do participante e sessão, "Salvar configuração" (envia `CONFIGURE`). Principal: botões Sincronizar / PLAY / Iniciar NFB (com confirmação) / STOP, indicadores `trigger/aurora/channels`, qualidade por canal, faixa de alerta de failsafe, resumo da sessão. Polling do estado a cada 300 ms.
- **Dependências:** `tkinter`, `pylsl`, `session_ipc`, `acquisition.StreamMap`. Textos da interface em português sem acentos.

`session_ipc.py`

- **Propósito:** IPC por arquivo JSON local (`run/nfb_state.json`, `run/nfb_command.json`).
- `write_state`, `read_state`, `send_command(cmd_id, cmd, payload)`, `read_command`, `reset`; `_atomic_write` (arquivo temporário + `os.replace`, até 6 tentativas com espera crescente para absorver lock momentâneo do OneDrive); `_read_json` trata arquivo ausente/corrompido como "sem novidade".
- **Quem chama:** `control_panel.py`, `nfb_process.py`, testes.

`run_neurofeedback.py`

- **Propósito (duplo):** (a) define `SignalWorker`, usado pela aplicação de produção; (b) é o runner autônomo das Camadas 2–5, sem painel (herdado da fase de replay).
- `SignalWorker(threading.Thread)` — Camadas 2+3 em thread própria: `acq.drain` → `pipeline.add_sample` sob `lock`; escuta opcional de um stream de markers (`pending_markers`, `take_markers`); `stop`.
- `run(duration, display, stream_name, log_prefix, fps, quiet)` — laço principal do runner autônomo. Argumentos de linha de comando: `--duration`, `--no-display`, `--stream`, `--prefix`.
- No fluxo de produção, `take_markers()` não é chamado por `NfbSession`; os markers do log vêm de `_pending_log_marker`.

2.3 Mocks e utilitários de replay

`mock_aurora_lsl.py`

- **Propósito:** servidor de replay LSL que imita o Data Out do Aurora a partir do cache `replay_hb.npz` (derivado do `.nirs` de teste, outra tarefa, montagem whole-head). Schema **antigo** (rótulos anatômicos em `source/detector`, `type = HbO/raw760`...).
- Funções: `build_stream_info`, `load_cache`, `run(speed, stop_after, loop, name, markers, verbose)`. CLI: `--speed`, `--stop-after`, `--loop`, `--name`, `--no-markers`.
- Publica também um stream `Trigger` (Markers) com os eventos do `.nirs` (por padrão).
- **Dependências:** `numpy`, `pylsl`, `nf_config`; requer `replay_hb.npz`.

`mock_aurora_lsl_synthetic.py`

- **Propósito:** mock de replay contínuo com dado sintético (ruído AR(1) + deriva senoidal lenta; não fisiológico) já no **schema real** (`nirs_hbo/nirs_hbr/nirs_raw+wavelength/nirs_frame`, `custom_name` `S{n}-D{n}`, `source/detector` vazios). 8 fontes, 15 detectores, 24 pares (16 longos + 8 curtos), 97 canais (4×24 + 1).
- Valida na importação que os pares de `PFC_CHANNELS` e `SHORT_CHANNELS_FRONTAL` existem na topologia sintética (`RuntimeError` caso contrário).
- Funções: `build_stream_info`, `_ChannelNoise`, `run(...)`. CLI igual ao `mock_aurora_lsl.py` (`--loop` é aceito mas sem efeito, geração infinita). Publica `Trigger` com valor `1` a cada ~5 s de dado, por padrão.
- **Dependências:** `pylsl`, `nf_config`. Não usa `mne`.

`nirs_to_hb.py`

- **Propósito:** converte o `.nirs` (Homer2) de teste em HbO/HbR (Beer-Lambert modificada, coeficientes de `cfg.EXTINCTION`, `cfg.DPF`) e grava o cache `replay_hb.npz`.
- Funções: `load_nirs`, `channel_table`, `intensity_to_od`, `od_to_conc`, `convert`. Usado apenas pelo mock antigo e pelos testes que o usam. O arquivo `.nirs` referenciado em `cfg.NIRS_FILE` não está na pasta do projeto (a confirmar disponibilidade; o cache `replay_hb.npz` está).
- **Dependências:** `numpy`, `scipy`, `nf_config`.

2.4 Testes

Detalhes e cobertura na seção 7.

Script	Função
`test_acceptance.py`	aceitação do replay ponta a ponta contra `mock_aurora_lsl.py` (sem display)
`test_live_connection.py`	caminho "ao vivo" de `Acquisition` (stream ausente / presente)
`test_session_state_machine.py`	máquina de estados do `NfbSession` e IPC, contra o mock sintético
`test_real_schema.py`	`StreamMap`, `Acquisition`, detecção de canais do painel e publicação de markers contra metadata sintética no schema real

2.5 Outros arquivos

- `iniciar_mock.bat` / `iniciar_neurofeedback.bat` — atalhos Windows que chamam `C:\Program Files\PsychoPy\python.exe` (caminho fixo no `.bat`) para `mock_aurora_lsl.py` e `run_neurofeedback.py --duration 90`. Este último é o runner autônomo legado; **não há `.bat` para a aplicação de produção** (`control_panel.py` + `nfb_process.py`).
- `INSTRUCOES_PERSONAGEM_TRILHA.md` — documento de referência (pygame) de onde a lógica da trilha foi portada.
- `Psychopy_TrialTypes.xlsx` — não analisado nesta documentação (a confirmar uso).
- `trail_bg.png`, `hiker_sprite.png` — assets visuais (na raiz do projeto).

3. Parâmetros de `nf_config.py`

Status: **calibrado** = valor validado com dado real; **placeholder** = valor inicial adotado, sem calibração em piloto; **informativo** = não afeta o cálculo. A classificação foi derivada da seção "Pendências" do `CLAUDE.md` e do código; onde não há evidência, "a confirmar".

Parâmetro	Valor atual	Significado	Status
`REPLAY_MODE`	`True`	Rótulo informativo gravado em log/metadados; não é gate de código	informativo
`BASE_DIR` / `LOG_DIR` / `RUN_DIR`	pasta do script / `logs` / `run`	Diretórios de trabalho	—
`NIRS_FILE`	`2025-07-09_003.nirs` na pasta do projeto	`.nirs` de teste para `nirs_to_hb.py` (arquivo ausente da pasta; o `CLAUDE.md` cita `2025-04-02_004.nirs`)	a confirmar
`HB_CACHE`	`replay_hb.npz`	Cache de HbO/HbR para o mock antigo	—
`STREAM_NAME`	`"Aurora"`	Nome do stream Data Out	calibrado (confirmado na tela do Aurora, segundo `CLAUDE.md`); o comentário "PENDENCIA" no código está desatualizado
`STREAM_TYPE`	`"NIRS"`	Tipo do stream publicado pelos mocks	informativo
`STREAM_SOURCE_ID`	`"mock_aurora_replay"`	`source_id` dos mocks	informativo
`SERIAL_NUMBER`	`"MOCK-NSP2-0000"`	Número de série fictício dos mocks	informativo
`SOURCE_LABELS` / `DETECTOR_LABELS`	rótulos 10-20 do `.nirs` de teste	Usados só por `nirs_to_hb.py` (mock antigo)	informativo
`PFC_CHANNELS`	`S1-D1, S1-D2, S2-D1, S2-D2, S3-D3, S3-D4, S4-D3, S4-D4`	Pré-seleção do ROI (8 canais). Escolha por geometria; **não confirmada anatomicamente**	placeholder (pré-seleção; seleção final manual no painel)
`SHORT_CHANNELS_FRONTAL`	`S1-D8, S2-D9, S3-D10, S4-D11`	Canais curtos frontais. Não são usados no feedback; usados só na validação do mock sintético	placeholder

Parâmetro	Valor atual	Significado	Status
EXTINCTION	{760: (1486.5865, 3843.707), 850: (2526.391, 1798.643)}	Coeficientes (HbO, HbR) usados em nirs_to_hb.py	informativo (conversão offline do mock antigo)
DPF	6.0 / 6.0	Fator de caminho diferencial (mock antigo)	informativo
SHORT_CHANNEL_MAX_CM	1.5	Limite de distância para classificar canal curto (mock antigo)	informativo
BASELINE_S	30.0	Duração do baseline (s), em tempo do dispositivo	placeholder
FEATURE_WINDOW_S	4.0	Janela deslizante da média de HbO (s)	placeholder
SMOOTH_TAU_S	1.5	Constante de tempo da suavização exponencial (s)	placeholder
NORM_CLIP	3.0	Saturação do z-score em ±3	placeholder (saturação observada em teste; reavaliar com dado real)
FAILSAFE_TIMEOUT_S	1.0	Tempo sem amostra válida (relógio de parede) para ativar o failsafe	placeholder
GLIDE_TAU_S	2.0	Constante de tempo do "planeio" durante o failsafe	placeholder
TRIGGER_SOURCE_ID	"nfb_process_trigger"	source_id do outlet Trigger	—
MARKER_BASELINE_START / MARKER_REGULATION_START / MARKER_STOP	1 / 2 / 3	Códigos dos markers	—
ALTITUDE_RANGE	(-0.75, 0.75)	Faixa interna da saída do pipeline (nome herdado da metáfora do avião); reescalada para 0–1 em trail_display.py	—
PLANE_X_SPEED	0.0	Não referenciado por nenhum script (resíduo da metáfora anterior)	informativo
WINDOW_SIZE / FULLSCREEN	(1280, 720) / False	Janela do PsychoPy	—

Constantes fora de nf_config.py: SMOOTHING_SPEED = 1.2 e HIKER_HEIGHT_NORM = 0.48 (trail_display.py, suavização adicional e tamanho do sprite); PLAUSIBLE_HBO_ABS_MAX = 200.0 (nfb_process.py); POLL_MS = 300 (control_panel.py); poll_hz = 30 (NfbSession.run_forever).

4. Schema real da metadata do Aurora e interpretação pelo StreamMap

Confirmado em diagnóstico de campo e no manual Aurora fNIRS 2025.2.5, seção 8.1.1 (segundo CLAUDE.md): 97 canais = 8 fontes, 15 detectores, 24 pares (16 longos + 8 curtos); colunas na ordem [24 raw 760 nm] + [24 raw 850 nm] + [24 HbO] + [24 HbR] + [1 frame/timestamp]. O Aurora entrega HbO/HbR já calculados (µmol/L); o pipeline não reprocessa.

Campo da metadata (por canal)	Valor real	Interpretação no StreamMap
type	nirs_hbo / nirs_hbr / nirs_raw / nirs_frame	traduzido para HbO / HbR / raw(760/850) / timestamp
wavelength	760.0 / 850.0 (só em nirs_raw)	decide raw760 vs raw850
custom_name	S{n}-D{n}	regex S(\d+)-D(\d+) → source="S{n}", detector="D{n}"
label	{src}-{det}:{idx}	fallback (regex (\d+)-(\d+):\d+) quando custom_name está ausente

Campo da metadata (por canal)	Valor real	Interpretação no `StreamMap`
`source`, `detector`	vazios	se vazios, são preenchidos a partir de `custom_name/label`
`short_channel`	`"true"` / `"false"`	`short=True` se `"true"`
`unit`	`umol/L` etc.	armazenado em `unit`, não usado no cálculo

Seleção do ROI: `pfc_hbo_columns(pairs)` busca, para cada par `(S, D)`, **exatamente uma** coluna `HbO`; se algum par não for encontrado, levanta `RuntimeError("canais PFC ausentes na metadata: ...")`. Coluna de tempo: `timestamp_column()` devolve a primeira coluna `nirs_frame`.

A confirmar: `Acquisition.pull` usa o valor da coluna `nirs_frame` como `device_ts`, e o pipeline mede `baseline_s` e `window_s` nesse eixo. O código assume que esse valor está em segundos. Se no Aurora real essa coluna for contador de quadros (e não segundos), as janelas ficariam em unidades de amostras. O mock sintético grava segundos. Verificar com sessão real (`device_time` no CSV deve crescer ≈ 1 por segundo).

Saída bruta do `StreamMap` usada no meta de sessão: `stream_name`, `stream_sfreq`, `n_stream_channels`, `pfc_pairs`, `pfc_hbo_columns`, `pfc_labels`.

5. Máquina de estados da sessão

Há dois níveis: a **fase** (`NfbSession.phase`, publicada no IPC e mostrada no painel) e o **estado do pipeline** (`FeaturePipeline.state`, gravado na coluna `state` do CSV).

```
idle --SYNC (todos os checks OK)--> synced --PLAY--> baseline --START_NFB--> regulation
  ^         |                                          |                         |
  +--SYNC (falha) volta/permanece idle                 +----------STOP-----------+--> stopped
```

- `SYNC`: verifica (1) stream `Markers` resolvido em 2 s; (2) stream Data Out resolvido em 3 s e com amostra em 3 s; (3) canais do ROI presentes na metadata, com checagem simples de qualidade (`|valor| < 200 µmol/L` e não-NaN, em uma única amostra). Se todos OK → `synced`; senão → `idle`.
- `PLAY` (só em `synced`): cria `Acquisition`, `FeaturePipeline(manual_gate=True)`, `Logger`, inicia o `SignalWorker`, emite marker `1` e entra em `baseline`.
- `START_NFB` (só em `baseline`): chama `pipeline.begin_regulation()`, muda a fase para `regulation` e emite marker `2`.
- `STOP` (qualquer fase): emite marker `3`, para o worker, fecha aquisição e log, produz o resumo e vai para `stopped`.
- Esc na janela PsychoPy também executa o stop.

Estado do pipeline: `baseline` → (após `BASELINE_S` de dado) `baseline_ready` → (após `begin_regulation`) `running` ↔ `failsafe`. **`baseline_ready` não é uma fase** do `NfbSession`; só aparece no CSV, no snapshot e no `state` publicado.

Papel do gate manual: com `manual_gate=True`, ao completar o baseline o pipeline **não** avança sozinho; fica em `baseline_ready` (a trilha permanece parada) até `begin_regulation()`. No runner autônomo (`run_neurofeedback.py`) o gate está desligado (`manual_gate=False`) e o pipeline passa direto para `running`.

Atenção (ver limitações): o botão "Iniciar NFB" fica habilitado durante toda a fase `baseline`, não só em `baseline_ready`; `_do_start_nfb` troca a fase para `regulation` e emite o marker `2` mesmo se o pipeline ainda estiver coletando baseline (nesse caso `begin_regulation()` não tem efeito).

5.1 Failsafe

Em `update_display`, se não houver amostra válida há mais de `FAILSAFE_TIMEOUT_S` (relógio de parede) e o pipeline não estiver em `baseline`/`baseline_ready`, o estado vira `failsafe`: a suavização para (`smoothed` fica congelado) e a altitude "planeia" em direção ao valor congelado (`GLIDE_TAU_S`). A trilha mantém a última posição válida e mostra o aviso "SINAL PERDIDO - mantendo posicao". Quando amostras voltam, `add_sample` retorna o estado para `running`. O painel

exibe um banner persistente se o failsafe foi acionado na sessão. Durante o baseline não há failsafe: se o sinal parar, o baseline apenas deixa de avançar.

6. Como rodar

Use sempre o Python embutido no PsychoPy (onde `pylsl`, PsychoPy, PIL, numpy estão instalados), não o Python do sistema. Abaixo, `PSY = "C:\Program Files\PsychoPy\python.exe"` (caminho de instalação padrão; ajustar se diferente). A partir da pasta do projeto:

```
# Aplicacao de producao (dois terminais/processos)
PSY nfb_process.py          # Processo B: janela de neurofeedback (reseta a pasta run/ ao iniciar)
PSY control_panel.py        # Processo A: painel de controle

# Mock sintetico (schema real), sem hardware
PSY mock_aurora_lsl_synthetic.py --speed 1 [--stop-after 20] [--no-markers]

# Mock antigo (precisa de replay_hb.npz; gerar com nirs_to_hb.py se ausente)
PSY nirs_to_hb.py
PSY mock_aurora_lsl.py --speed 1 [--loop] [--stop-after N] [--no-markers]

# Runner autonomo (sem painel)
PSY run_neurofeedback.py --duration 90 [--no-display] [--stream NOME] [--prefix PREFIXO]
```

Ordem para sessão real: iniciar `nfb_process.py` primeiro (ele zera `run/`), depois `control_panel.py`; Aurora com stream Data Out ativo → aba Configurações: "Detectar canais", conferir o ROI, preencher ID/sessão, "Salvar configuração" → aba Principal: Sincronizar (3 indicadores OK) → PLAY → aguardar fim do baseline → Iniciar NFB (confirmar) → STOP. Checklist operacional de conexão ao equipamento: a confirmar (mantido fora do código, segundo `CLAUDE.md`).

Para mock + aplicação: rode `mock_aurora_lsl_synthetic.py` e depois `nfb_process.py`/`control_panel.py`; use `--no-markers` no mock para não criar um segundo stream `Trigger` (ver Troubleshooting).

7. Testes

Todos usam o Python do PsychoPy (`PSY nome_do_teste.py`) e retornam código 0/1 com linhas `[OK]`/`[FALHA]`. Testes que sobem mock próprio o fazem como subprocesso. **Não rode dois testes (ou um teste e um mock manual) ao mesmo tempo.**

Teste	Mock usado	Cobertura	Situação
`test_session_state_machine.py`	`mock_aurora_lsl_synthetic.py`	CONFIGURE via IPC com `PFC_CHANNELS` real → SYNC → PLAY (baseline) → baseline_ready sem avanço automático → tela parada → START_NFB → regulation/running → STOP, resumo, log com `baseline_ready` e `running`, meta com participante/sessão/ROI (9/9 segundo `CLAUDE.md`)	válido para a montagem real
`test_real_schema.py`	metadata sintética própria (stream próprio, 97 canais)	`StreamMap` (24 pares HbO/HbR, `raw760` vs `raw850` por `wavelength`, `nirs_frame` → timestamp, 8 curtos, fallback pelo `label`), `Acquisition` com ROI arbitrário, worker de detecção do painel, SYNC com ROI vazio, publicação do `Trigger` (19/19 segundo `CLAUDE.md`)	válido; não depende de `PFC_CHANNELS`

Teste	Mock usado	Cobertura	Situação
`test_acceptance.py`	`mock_aurora_lsl.py` (`.nirs` whole-head, schema antigo)	mapeamento do ROI pela metadata, baseline, progresso contínuo e sem saltos, failsafe ao parar o mock, planeio sem queda, não recua, log consistente, timestamps de dispositivo, markers do mock	**falha contra a montagem real por incompatibilidade de dataset — esperado**
`test_live_connection.py`	`mock_aurora_lsl.py`	stream ausente → `RuntimeError` limpo; stream presente com `REPLAY_MODE=False` → conecta e mapeia o ROI padrão	**falha contra a montagem real por incompatibilidade de dataset — esperado**

Os dois últimos usam `cfg.PFC_CHANNELS` (formato `S{n}-D{n}`) contra o mock antigo (rótulos anatômicos, p. ex. AF4/AF8 ausentes no `.nirs` whole-head), então o `RuntimeError: canais PFC ausentes na metadata` é esperado. Servem hoje apenas para a engenharia genérica; não validam a montagem do experimento. Os resultados acima foram registrados no `CLAUDE.md` e **não foram reexecutados** para esta documentação.

Não há teste automático para: `trail_display.py` (visual validado manualmente), `nf_logger.py` isoladamente, `mock_aurora_lsl.py` isoladamente, nem para o comportamento do `START_NFB` antes de `baseline_ready`.

8. Troubleshooting

Sintoma	Causa conhecida	O que fazer
Baseline incompleto, trilha travada, failsafe não ativa; leitura estranha	**Dois mocks (ou mock + outro publicador) com o mesmo nome de stream `Aurora`**: competem e corrompem a leitura	Encerrar todos os processos de mock antigos antes de subir um novo; não rodar `test_*.py` com mock próprio em paralelo com outro mock
SYNC acha o stream `Trigger` errado / dois `Trigger`	Mocks publicam `Trigger` por padrão e o `nfb_process.py` também publica `Trigger`	Subir o mock com `--no-markers` ao usar a aplicação de produção. `resolve_byprop("type","Markers")` pega o primeiro que encontrar
`ModuleNotFoundError: pylsl/psychopy` ou comportamento diferente entre máquinas	Executou com o Python do sistema (ou do instalador do MNE) em vez do Python do PsychoPy	Usar `C:\Program Files\PsychoPy\python.exe`; reinstalar `pylsl`/`mne`/`mne-nirs`/`pygame` no Python embutido do PsychoPy ao trocar de máquina/versão e reexecutar os testes
`PermissionError` / `WinError 5` ao escrever em `run/`	OneDrive segura lock momentâneo no arquivo	Já mitigado: `_atomic_write` tenta até 6 vezes. Se persistir, mover o projeto para fora da pasta sincronizada
Após "Detectar canais" o ROI ficou menor ou vazio; SYNC mostra "N/8 canais" ou "nenhum canal selecionado"	A detecção substitui a lista pelas HbO do stream encontrado; só ficam pré-marcados os pares que coincidem com `PFC_CHANNELS`. Se o stream detectado era o errado (ex.: mock antigo com rótulos anatômicos), nenhum/poucos pares coincidem	Conferir o nome do stream e qual publicador está ativo, rodar "Detectar canais" de novo, remarcar o ROI e "Salvar configuração"
Mudanças no ROI/ID não surtem efeito	A configuração só vai ao processo B ao clicar "Salvar configuração" (`CONFIGURE`); sem isso valem os padrões	Salvar a configuração antes de Sincronizar
SYNC: "stream resolvido mas sem amostras em 3s"	Stream existe mas não emite (no Aurora, aquisição/gravação (Record) não ativa — relatado em campo)	Iniciar o Record no Aurora e sincronizar de novo
SYNC: "stream 'Aurora' nao encontrado em 3s"	Nome diferente, Aurora sem LSL Data Out, ou mock não iniciado	Conferir o nome na tela de LSL Stream Names do Aurora e o campo do painel
Painel reiniciado e botões não têm efeito	O contador de comandos do painel recomeça em 1; o `nfb_process.py` ignora	Reiniciar os dois processos juntos (começando por `nfb_process.py`)

Sintoma	Causa conhecida	O que fazer
	ids ≤ último processado	
PNG sem transparência (xadrez no sprite/fundo)	Geradores de imagem às vezes pintam xadrez falso nos pixels	Remover o fundo de verdade (ex.: `rembg`) antes de usar

9. Latência e throughput do sistema

Rótulos de origem: **[Aurora]** informado pelo Aurora (metadata do stream ou manual); **[código]** derivado do código/config, com a conta mostrada; **[medido]** medido nesta tarefa, na máquina de desenvolvimento (não a do laboratório); **[a confirmar]** sem dado, sem estimativa.

Máquina das medições [medido]: PC de desenvolvimento, Windows 10 Pro (10.0.19045), CPU AMD64 Family 23 (16 processadores lógicos), Python 3.8.10 do PsychoPy 2022.13, `pylsl` 1.15.0 / liblsl 1.15.0, contra `mock_aurora_lsl_synthetic.py` em loopback (mesma máquina), sem hardware Aurora e sem janela PsychoPy. Os resultados **não** se transferem para a máquina do laboratório nem para o PsychoPy 2026.1.3 sem nova medição.

Informações reportadas pelo Aurora

Item	Valor	Origem	Observação
Taxa nominal	≈ 10,17 Hz	[Aurora]	Informado a partir da metadata capturada em campo. No repositório aparece como `SFREQ = 10.17` em `mock_aurora_lsl_synthetic.py` (comentário: "cadência documentada") e como ≈ 10,2 Hz no `CLAUDE.md`. O valor exato é lido de `info.nominal_srate()` a cada sessão e gravado em `stream_sfreq` no `*_meta.json`
`steps_per_frame`	8	[Aurora]	Informado pelo pesquisador (metadata de campo). Não consta em nenhum arquivo da pasta e não é usado pelo código. Significado exato: [a confirmar]
Nº de colunas do stream	97	[Aurora]	1 frame + 24 canais × (raw760, raw850, HbO, HbR) = 1 + 96. Corroborado por `test_real_schema.py` e pelo mock sintético (`N_OUT = 4×24+1`)
Formato dos dados	float32	[Aurora]	Informado pelo pesquisador; os mocks publicam `double64`. Não verificado no código
Hardware, versão do Aurora, `ntrigger_inputs`	—	[a confirmar]	Valores não disponíveis na pasta
Fallback de trigger	12	[Aurora]	Informado no `CLAUDE.md` (tela LSL Stream Names do Aurora)
Latência de saída do stream	—	[a confirmar]	O manual PDF do Aurora e o script `diagnostico_metadata.py` não estão na pasta; nenhuma latência interna do Aurora foi deduzida

Throughput

Grandeza	Valor	Origem	Conta / observação
Período de amostragem	98,3 ms	[código]	1 / 10,17 Hz
Amostras/s do stream	10,17	[Aurora]	taxa nominal
Valores/s do stream completo	≈ 986	[código]	97 colunas × 10,17
Valores/s usados pelo pipeline	≈ 81	[código]	8 colunas HbO do ROI × 10,17. A amostra completa (97 valores) é recebida e convertida em array; só as colunas do ROI entram em `add_sample`
Amostras novas por quadro de tela	≈ 0,17	[código]	10,17 / 60 fps, ou seja, uma amostra nova a cada ≈ 5,9 quadros
Linhas de log por segundo (`nfb_process.py`)	≈ 30 (alvo)	[código]	uma linha por `tick`; laço com `poll_hz = 30`. Em um log de mock (123 linhas em ≈ 3,6 s) observaram-se ≈ 34 linhas/s [medido nos logs existentes]; taxa em sessão de laboratório: [a confirmar]
Linhas de log por segundo (`run_neurofeedback.py`)	≈ taxa de quadros (≈ 60)	[código]	uma linha por iteração; sem display, `fps = 60`
Limite de `drain` por ciclo	64 amostras	[código]	`SignalWorker`: `drain(max_samples=64, timeout=0.05)`; muito acima de 10,17 amostras/s
Buffer do inlet	60 s	[código]	`StreamInlet(max_buflen=60)`
Taxa do laço principal (`nfb_process.py`)	30 Hz	[código]	`run_forever(poll_hz=30)`; o `flip` do PsychoPy pode limitar à taxa do monitor; taxa real: [a confirmar]
Polling do painel	300 ms	[código]	`POLL_MS = 300` (leitura de `nfb_state.json`)

Orçamento de latência por etapa (da chegada da amostra ao pixel)

Etapa	Valor	Origem	Observação
Período de amostragem do Aurora	98,3 ms	[código]	1 / 10,17 Hz; um evento só aparece na próxima amostra (0–98 ms, média ≈ 49 ms). Se cada amostra é média sobre o intervalo: [a confirmar]
Latência interna do Aurora (aquisição → saída LSL)	—	[a confirmar]	sem dado
Transporte LSL (mock em loopback)	média 1,15 ms; DP 0,47 ms; p95 1,80 ms; máx. 5,60 ms (n = 306)	[medido]	`local_clock()` no recebimento − timestamp LSL da amostra. O timestamp é o instante agendado pelo mock, então inclui o jitter do agendamento do mock. **Não** representa o ambiente do laboratório nem o Aurora real (transporte real: [a confirmar])

Etapa	Valor	Origem	Observação
Processamento por amostra	`add_sample`: média 0,054 ms (p95 0,059; máx. 0,161). `update_display` + `snapshot`: média 0,011 ms (máx. 0,085). `Logger.write`: média 0,040 ms (máx. 0,193)	[medido]	n = 306 amostras em 30 s; log gravado em pasta temporária local, não na pasta do projeto (OneDrive). Custo do IPC (`write_state`, ≈ 30×/s, em pasta OneDrive) e do desenho PsychoPy: [a confirmar]
Coleta pelo `SignalWorker`	≈ imediata após a chegada	[código]	`pull_sample(timeout=0.05)` bloqueia e retorna assim que há amostra; o timeout só limita a espera sem dado (não é polling cego de 50 ms). Atraso incluído na linha do transporte
Laço principal lê o estado	0–33 ms (média ≈ 17 ms)	[código]	`poll_hz = 30`: o `tick` usa o estado do pipeline na iteração seguinte
Janela deslizante (média simples)	atraso de grupo ≈ 1,97 s (≈ `FEATURE_WINDOW_S`/2)	[código]	`np.mean` sobre as amostras com `t − t_mais_antiga ≤ 4,0 s`: N = ⌊4,0 × 10,17⌋ + 1 = 41 amostras. Média simples (boxcar) tem atraso de grupo (N−1)/2 = 20 amostras = 20 / 10,17 ≈ 1,97 s. Resposta a degrau: rampa de 0 a 100% em 4,0 s (50% em ≈ 2,0 s)
Suavização exponencial (`SMOOTH_TAU_S` = 1,5 s)	63% em 1,5 s; 95% em ≈ 4,5 s (3τ)	[código]	1 − e^(−t/τ); atraso de grupo em baixa frequência ≈ τ = 1,5 s. Atualizada a cada `tick` com `dt` real (independe da taxa)
Suavização do progresso no `trail_display` (`SMOOTHING_SPEED` = 1,2)	τ ≈ 0,83 s: 63% em ≈ 0,83 s; 95% em ≈ 2,5 s	[código]	`progresso += (alvo − progresso) × min(1, 1,2 × dt)` → τ ≈ 1 / 1,2 s^{-1} (aproximação válida para 1,2·dt ≪ 1)
Período de quadro / `flip`	≈ 16,7 ms a 60 Hz	[código]	`win.flip()` em `TrailDisplay.draw`. ≈ 59,3 fps foi registrado em teste anterior (`CLAUDE.md`); latência do monitor/SO: [a confirmar]
Timeout do failsafe (`FAILSAFE_TIMEOUT_S`)	1,0 s (+ até 33 ms do laço)	[código]	≈ 10 períodos de amostragem sem amostra válida (relógio de parede)
Resposta hemodinâmica	—	[a confirmar]	Atraso fisiológico da ordem de segundos, fora do controle do sistema: [REF: hemodynamic response latency]

Estimativa do atraso determinístico do pipeline (chegada da amostra ao pixel), somente itens [código]/[medido]: os três filtros em cascata somam atrasos de grupo (aproximação para sinais lentos): 1,97 s (janela) + 1,5 s (EMA) + ≈ 0,83 s (progresso na trilha) ≈ **4,3 s**, mais um termo fixo ≤ ≈ 0,05 s (≈ 1 ms de transporte/coleta [medido], até 33 ms do laço e até 17 ms de quadro). É um atraso de grupo (centróide da resposta ao impulso), não o tempo para refletir 100% de uma mudança: a janela leva 4,0 s para refletir um degrau por completo e a EMA e a suavização do desenho têm cauda exponencial. O z-score e a saturação (`±NORM_CLIP`) não adicionam atraso, mas a saturação trunca excursões.

Fica de fora da estimativa: latência interna do Aurora e do hardware, transporte LSL no ambiente real, quantização pelo período de amostragem (0–98 ms), custo do desenho PsychoPy e do IPC, latência do monitor e a resposta hemodinâmica ([REF: hemodynamic response latency]).

Medição auxiliar nos logs existentes (agregada)

[medido, logs em `logs/`, apenas estatísticas agregadas] Nos 6 logs de mock em velocidade 1× (≈ 15,4 mil linhas), a diferença `lsl_time − device_time` é um deslocamento entre dois relógios arbitrários (relógio LSL × tempo do dispositivo), não uma latência; o que importa é sua estabilidade. Em torno da média de cada log: DP agregado = 0,84 s e desvio máximo = 16,7 s; a faixa (máx.−mín.) por log tem mediana 0,000 s e máximo 32,0 s. Os valores altos vêm de poucos logs em que o deslocamento muda no meio da execução (causa: a confirmar; hipótese: `--loop` ou reinício do mock); nos demais o deslocamento é constante. Os 13 logs em velocidade 10× foram excluídos, pois o tempo do dispositivo e o relógio LSL avançam em ritmos diferentes. Medição com o Aurora real: [a confirmar].

10. Limitações conhecidas

1. **Gate manual incompleto:** "Iniciar NFB" é habilitado em toda a fase `baseline`. Se acionado antes de `baseline_ready`, a fase vai para `regulation` e o marker `2` é emitido, mas o pipeline continua em baseline; ao terminar fica em `baseline_ready` sem que o botão volte a estar habilitado (a confirmação no diálogo é a única proteção). Deduzido do código; não reproduzido nesta etapa.
2. **Eixo de tempo do dispositivo:** `nirs_frame` tratado como segundos (a confirmar com o Aurora real, seção 4).
3. **Qualidade de sinal simplificada:** `|valor| < 200 µmol/L` e não-NaN, avaliada numa única amostra durante o SYNC; sem SNR, sem monitoramento contínuo, sem indicador nativo do Aurora (verificar se existe na metadata antes de coleta).
4. **Parâmetros não calibrados:** janela, suavização, `NORM_CLIP` (saturação observada), duração do baseline.
5. **ROI por geometria:** cluster S1–S4 não confirmado anatomicamente; seleção final manual.
6. **Canais curtos:** não são usados na feature nem há correção de sinal sistêmico implementada.
7. **Dupla suavização:** EMA no pipeline (`SMOOTH_TAU_S`) e suavização do progresso no `trail_display.py` (`SMOOTHING_SPEED`).
8. **Markers duplicados/colisão:** mocks usam `Trigger` com valor `1` (mesmo código do início de baseline); com um mock e o `nfb_process.py` ativos há dois streams `Trigger`.
9. **Fila de markers não consumida:** no `nfb_process.py`, o `SignalWorker` acumula `pending_markers` de um stream externo que nunca é lido (crescimento de lista; irrelevante para sessões curtas).
10. **Validação:** apenas testes de engenharia com mock/dado sintético, mais as primeiras sessões de teste no equipamento; sem validação de desempenho do neurofeedback.
11. **Dataset de teste divergente:** `test_acceptance.py` e `test_live_connection.py` falham contra a montagem real (seção 7).
12. **Cópias do projeto:** existiram versões divergentes entre PC pessoal e PC do lab; a regra é editar só no PC pessoal e copiar a pasta para o PC do lab.
13. **Plataforma:** interface e testes validados em Windows; caminhos de `.bat` fixos.
14. **Idioma:** interface (tkinter/PsychoPy), mensagens e comentários do código estão em português sem acentos.
15. **Assets:** `trail_bg.png` tem ≈ 6,2 MB.
16. **Latência:** sem medição com o Aurora real (seção 9). Os mocks publicam `double64`; o Aurora real, segundo informado, `float32`.